

MISE EN ŒUVRE D'UN PROCESSUS DE DATA MINING

Détection des annonces susceptibles d'entraîner une modification de la base vectorielle de l'IGN

Adrien Guille - SAE5.03 - Université Lumière Lyon 2

Objectif et rendu

L'objectif est de mettre au point le meilleur système pour distinguer automatiquement les annonces pertinentes des annonces hors spécifications parmi celles publiées dans le bulletin des annonces de marché public (BOAMP). La variable à prédire est nommée `cal_réponse_signalement`.

Le rendu attendu consiste en :

1. Un notebook (extension `.ipynb`), réalisé en binôme. Vous devrez y démontrer votre capacité à dérouler une démarche cohérente pour identifier un classifieur performant, votre compréhension des méthodes employées, ainsi que votre capacité à analyser pertinemment les résultats obtenus.
2. Au moins une soumission, par trinôme, à la compétition organisée pour la SAE (détails à venir).

Ci-dessous, les pistes à explorer pour concevoir et analyser le système.

Pistes

Le but de l'étude est d'abord d'explorer le plus possible les capacités des classifieurs étudiés en cours. Si vous souhaitez sortir des pistes mentionnées ci-dessous, il faudra d'abord en parler avec l'enseignant responsable.

- **Préparation des données**

- Pré-traitement du texte :
 - Racinisation : `SnowballStemmer` (NLTK)
 - Lemmatisation : `Lemmatizer` (spaCy)
- Vectorisation du texte :
 - Comptages simples : `CountVectorizer` (scikit-learn)
 - Comptages pondérés : `TfidfVectorizer` (scikit-learn)
- Réduction de la dimension :
 - Décomposition tronquée en valeurs singulières : `TruncatedSVD` (scikit-learn)
 - Factorisation en matrices non négatives : `NMF` (scikit-learn)

- Construction de variables additionnelles :
 - Statistiques diverses à propos de la forme du texte : longueur, fréquence des classes grammaticales, présence d'URLs ou de nombres, etc.
- **Classification**
 - Classifieur bayésien naïf :
 - Multinomial : [MultinomialNB \(scikit-learn\)](#)
 - Gaussien : [GaussianNB \(scikit-learn\)](#)
 - Bernoulli : [BernoulliNB \(scikit-learn\)](#)
 - Régression logistique : [LogisticRegression \(scikit-learn\)](#)
 - Non régularisée
 - Régularisation : L1, L2, ElasticNet
 - Méthode des k plus proches voisins : [KNeighborsClassifier \(scikit-learn\)](#)
 - Arbre de décision : [DecisionTreeClassifier \(scikit-learn\)](#)
 - Forêt aléatoire : [RandomForestClassifier \(scikit-learn\)](#)
- **Analyse**
 - Visualisations diverses, notamment concernant les variables construites
 - Analyse thématique d'après la représentation en faible dimension des textes
 - Interprétation des classifieurs